Nesneye Dayalı Paradigma

23 Mart 2026 Pazartesi 10:07

ilk programlama dilleri ndp değildir.
90lı yıllarda nesneye yönelimlilik eklenmeye başlanmıştır.

c dili ndp desteklemez
ndp deskteleyen c dili olarak c++ piyasaya sürülmüştür

- Nesneye Dayalı Paradigmada;

mesaj alabilen mesaj gönderebilen yaşayan bir tür tanımlanmalıdır (nesne)

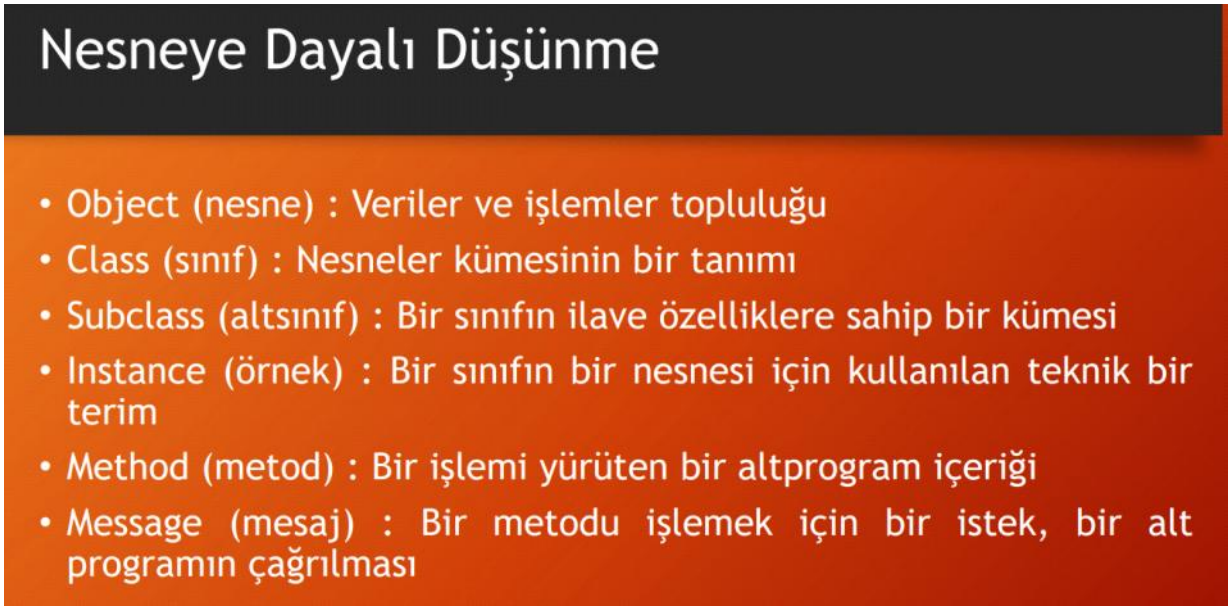
- simula67 ilk nesneye dayalı dildir. (simülasyon dili)
- Nesne yönelimli paradigma bir tasarım paradigmasıdır (programlama yapı taşıdır.)
- c++ hem emir esaslı hem de nesneye dayalı paradigmaya sahiptir.
- Java sadece nesneye dayalı paradigmadır
- C sadece emir esaslıdır

Şu 3 Temel Saç Ayağını içermeyen diller ndp değildir:

- Kalıtım
- Çok Biçimlilik
- Kapsülleme

GELENEKSEL DILLERDE:
Eskiden sorumluluklar fonksiyonlara yüklenirdi.
Dilin yapısı flowchart'a benzerdi.

NDP ILE:
Artık her sorumluluk bir sınıfa yükleniyor.
Varlık bağıntı diagramı çizmek kullanıcı tanımları yapmak şart.



Kapsülleme: sınıfın dışından bazı verilere erişilememesi ve değiştirilememesi.
Sadece ilgili olan veriler dışarıdan erişilebilir olmalıdır.

```
private int yas;

public getYas();
public setYas();

//get set metodları
```

Nesnenin Oluşturulması

- Sınıftan türetilecek bir nesnede içerisinde bulunacak elemanlar ilk değerlerini almaları gerekir.
- Bu durumda nesne oluşma aşamasında bir metot çağrılmalıdır.
- Bu fonksiyon kurucu veya yapıcı (constructor) fonksiyondur.

```
// Java   Ogrenci o1 = new Ogrenci("Ahmet Almaz",159);
class Ogrenci{
    private String isim;
    private int numara;

    public Ogrenci(String ism,int nmr){
        isim=ism;
        numara = nmr;
    }
}
```

```
//C++    Ogrenci o1("Ahmet Almaz",159);
class Ogrenci{
    private:
        string isim;
        int numara;
    public:
        Ogrenci(String ism,int nmr){
          isim=ism;
          numara = nmr;
        }
};
```

this Terimi

- Sınıf hiyerarşisinde, o anda oluşturulan nesneyi ifade etmek için kullanılır.
- Bazı durumlarda kullanımı gereklidir.
- Ayrıca okunabilirliği arttırmak için de kullanılmaktadır.

this teriminin kullanımı

```
class Kisi{
public String isim;
...

public Kisi(String isim){
    isim=isim;
    yas=0;
    boy=20;
    kilo=4;
  }
...
}
```

```
// doğrusu
public Kisi(String isim){
    this.isim=isim;
    yas=0;
    boy=20;
    kilo=4;
}
```

yanlış kod.
iki isim de aynı olduğundan kapsam yüzünden sadece parametre olan isim'i okuyabilir.
this.isim=isim olmalı

Nitelik Tanımı (Property)

- Nitelikler bazı programlama dillerinde desteklenmektedirler.
- Amaç sınıftaki bir veya birden fazla alt alanı okuma ve yazma yönünden korumaya almak veya yönetebilmektir.
- Nitelik desteklemeyen programlama dillerinde bu metotlar yardımıyla yapılır.

```
class Kisi
  {
    private double kilo;
    private double kalori;

    public Kisi()
    {
      kilo = 4;
      kalori = 0;
    }

// C# dili
```

```
public double Kalori
    {
      get
      {
        return kalori;
      }
      set
      {
        kalori = value;
    if (kalori > 1000)
      kilo += (kalori / 1000);
      }
    }
    public double Kilo
    {
      get
      {
        return kilo;
      }
    }
  }
```

C#'ta property desteği vardır direkt bir niteliğe get set yazılabilir.
Javada isimlendirmeyle get() set() metodu yazılır.
c++'da get fonku için getDegisken const{
}

şeklinde {} dan önce const yazılırsa o fonksiyon içinde veri değiştirilemez. (get fonk mantığı)

Yıkıcı Metotlar

- C++ dilinde programcı tarafından çağrılabilen yıkıcı metot java ve C#, Python gibi dillerde çöp toplayıcı tarafından çağrılır.

```
//Java
public class Arac{
  @Override
  protected void finalize() throws Throwable {
      // Çöp toplayıcı tarafından çağrılır.
  }
}
```

finakize() fonku yeni java sürümlerinde kaldırıldı

```
// C#
 class Arac : IDisposable
   {
      ~Arac()
      {
// Çöp toplayıcı tarafından
      }

      public void Dispose()
      {
// Programcı çağırabilir
// Nesnenin yok edilmesini garantilemez
      }
   }
```

```
// Python
class Arac:
   def __del__(self):
     # Çöp toplayıcı tarafından
```

Erişim Niteleyicileri

- Sınıflarda bilgi gizleme, dış kullanıma açma, kalıtım için kullanma gibi işlemler için erişim niteleyicileri kullanılır.
  - Varsayılan: Bazı dillerde
    - C++ (struct) public, C++(class) private
    - C# private
    - Java aynı pakette ise sınıf dışından erişilebilir.
  - private: Sadece sınıf içerisinden erişilebilir.
  - protected: Sınıf dışından sadece kalıtım yolu ile erişilebilir.
  - public: Sınıf içi ve dışında erişilebilir.

javada aynı pakette protectedlarda birbirine erişebilir.

Nesne Yaşam Döngüsü:

- Sınıfın Yüklenmesi -> static çalışacak şeylerin belirlenmesi
- Nesnenin Oluşturulması -> Kurucu func'ın çağrılıp örneklenmesi
- Nesnenin kullanımı (Yaşam)
- Nesnenin Yok Edilmesi

Kalıtım

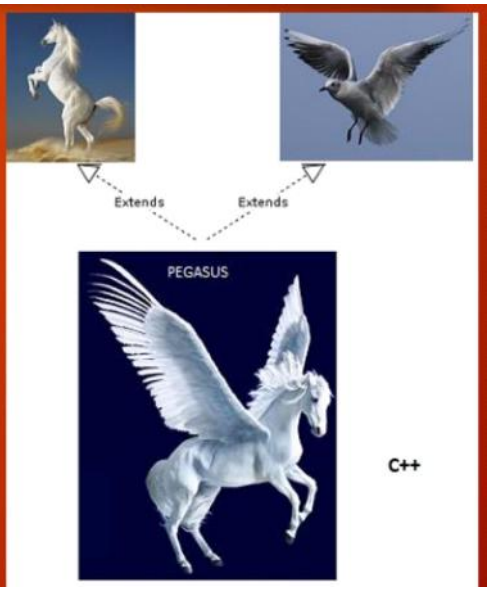
23 Mart 2026 Pazartesi 11:07

Kalıtım

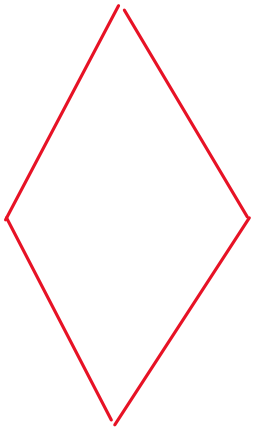
- Birçok tür tamamen bağımsız olmayacak ve bazı özellikleri başka türlere benzeyecektir.
- Bu durumda bu özelliklerin bir üst sınıftan alınması geliştirme sürecini hızlandırır.
- Kalıtım alınan sınıfa üst sınıf denir ve programlama dillerinde yukarıya mesaj gönderme şekli;
  - Java super
  - C# base
  - C++ Sınıf adı ile

c++ çoklu kalıtım içerdiğinden sınıf adı ile çağrılır.
python da çoklu kalıtım içerir.

- C++ gibi bazı dillerde çoklu sınıf kalıtımı desteklenir.
- Java ve C# dillerinde çoklu arayüz kalıtımı desteklenmektedir.
- Çok biçimlilik yine kalıtım ile desteklenmektedir.



Çoklu kalıtım Diamond problemine yol açabilir. Aynı isimde örneğin hem kuşta hem atta yemek ye func'ı var. O zaman ya override edicez ya da iki func'ın ismini değiştiricez (tavsiye edilmez Open/Closed princible'a aykırı)



```
class Canli{
        public:
                double kilo;
};

class Kus : public Canli{
        public:
                Kus(){
                        kilo=0.25;
                }
                void YemekYe(double kalori){
                        kilo += (kalori/10000);
                }
                double Kilo(){
                        return kilo;
                }
};
class At : public Canli{
        public:
                At(){
                        kilo=100;
                }
                void YemekYe(double kalori){
                        kilo += (kalori/1000);
                }
                double Kilo(){
                        return kilo;
                }
};
class Pegasus : public At, public Kus{
        public:
                Pegasus(){

                }
};
int main(){
        Pegasus *p = new Pegasus();
        p->YemekYe(1500);
        cout<<p->Kilo();
        delete p;
        return 0;
}
```

Overload (Aşırı Yükleme) ve Override (Ezme) Terimleri

- Overload bir metoda yeni anlamlar kazandırarak çok biçimliliği destekler.
- Override ise metodu yeniden tanımlamaktır. Dolayısıyla eski metodu ezmiş olacaktır.

C# dilinde operatörlerin aşırı yüklenmesi

```
class Sayi
  {
     public int deger;
     public Sayi(int dgr)
     {
        deger = dgr;
     }
     public static Sayi operator +(Sayi s1, Sayi s2)
     {
        return new Sayi(s1.deger+s2.deger);
     }
     public override string ToString()
     {
        return this.deger.ToString();
     }
  }
```

C#'ta ilkel türe benzer kullanım

```
static void Main(string[] args)
     {
        Sayi x = new Sayi(10);
        Sayi y = new Sayi(5);
        Console.WriteLine(x+y);
        Console.ReadKey();
     }
static void Main(string[] args)
     {
        int x = 10, y = 5;
        Console.WriteLine(x+y);
        Console.ReadKey();
     }
```